

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

6. Currificación

Funciones currificadas

```
unaFunc :: (Int, Int) -> Int  -- toma un par
unaFunc (x, y) = 2 * x + y

otraFunc :: Int -> Int -> Int  -- currificada
otraFunc x y = 2 * x + y
```

- Informalmente, ambas toman dos enteros.
- Formalmente:
 - La primera toma un argumento, que es un par de enteros.
 - La segunda toma un argumento `Int` y devuelve una función `Int -> Int`.
 - En este sentido, cada función en Haskell toma exactamente un argumento.
- Asociatividad:
 - `Int -> Int -> Int = Int -> (Int -> Int)`
 - `otraFunc 3 4 = (otraFunc 3) 4`
- La forma currificada es la preferida en Haskell.

Aplicación parcial

- `otraFunc 3` es una función (que toma un entero y le suma 6).

```
suma6 :: Int -> Int
suma6 y = otraFunc 3 y
```

- O incluso:

```
suma6' :: Int -> Int
suma6' = otraFunc 3
```

- Otros ejemplos:
 - `take 3 :: [a] -> [a]`
 - `elem 'a' :: String -> Bool`

Secciones de operadores

- Todas estas son formas válidas de escribir la función *sumar 1*:
 - `(+) 1`, `(+1)`, `(1+)`
 - Las dos últimas se llaman *secciones* del operador `+`.
 - `(+) 1` es el operador `+` convertido en función y aplicado parcialmente.
- Se pueden usar secciones con cualquier operador binario...
 - excepto la diferencia: `(-4)` representa el número `-4`...
 - aunque `(4-)` y `(-) 4` sí valen.
- Con operadores no conmutativos, las secciones tienen significados distintos:
 - `(/2) 8` es `4.0` y `(2/) 8` es `0.25`;
 - `(^2) 3` es `9` y `(2^) 3` es `8`.
- Las secciones también valen con operadores entre comillas invertidas: `(`div` 2)`.

Ejercicio

Haskell tiene definidas las funciones `curry` y `uncurry`.

```
| curry    :: ((a, b) -> c) -> a -> b -> c    -- de par a currificada
| uncurry  :: (a -> b -> c) -> (a, b) -> c    -- de currificada a par
```

Pueden ser usadas así:

```
| curry unaFunc 3 4
| uncurry otraFunc (3, 4)
```

1. Escribe la definición de esas dos funciones. Tienes que darles nombres distintos para que Haskell no se queje.
2. ¿Qué es `curry fst 7`?